Undergraduate Thesis

Implementation of an Efficient Distributed Storage System Based on two-tone Codes

Jiaqiang Yang

Student ID: 210110123

Supervisor: Associate Professor Ximing Fu

Major: Computer Science and Technology

School of Computer Science and Technology, Shenzhen Campus

Defense date: May 2026

Harbin Institute of Technology
May 2026

Abstract

Cloud computing, big data analytics, and artificial intelligence require distributed storage systems to balance reliability, capacity utilization, and access performance. Ceph provides mature object, block, and file storage services, but its mainstream erasure coding (EC) implementations rely primarily on finite-field matrix operations and incur substantial encoding and decoding costs. two-tone codes use Shift-XOR operations to simplify computation and offer high theoretical encoding efficiency. This thesis integrates two-tone erasure coding into Ceph, adapts the underlying data paths, and optimizes its performance.

We first analyze Ceph's EC plugin framework and its object write, read, and recovery paths. The main integration obstacle is variable-length parity chunks: some two-tone parity chunks exceed the length of data chunks, violating the equal-sized fragment assumption embedded in the Ceph OSD layer. We address this mismatch with actual-length awareness, block-aligned padding, and logical trimming during recovery. We add a parity-length query interface to the plugin, use actual fragment lengths for slicing and length assertions in encoding and recovery, and transparently pad fragments at the storage layer. These changes enable the complete workflow of writes, reads, benchmarking, and failure recovery.

We implement a general two-tone encoder and decoder that supports flexible parameter combinations and precomputes the shift layout to avoid repeated runtime calculations. We then optimize the decoding hot path through a single-fragment fast recovery path, tile-based cache-friendly decoding, loop reordering, temporary buffer reuse, and AVX2 SIMD XOR operations. In core microbenchmarks with chunk=2048 KB, two-tone achieves encoding throughputs of 11.49, 7.75, and 7.60 GB/s and decoding throughputs of 17.89, 13.67, and 13.23 GB/s for k=6,m=2, k=6,m=3, and k=8,m=3, respectively. It substantially outperforms Jerasure Reed-Solomon and exceeds ISA-L across these three configurations. In system-level recovery experiments, its average recovery bandwidth reaches 236, 123, and 124 MB/s, respectively, improving on Jerasure RS by 45.7%-53.8% and exceeding ISA-L on this bandwidth metric.

Keywords: Ceph; two-tone; erasure coding; distributed storage; performance optimization

Contents

Chapter 1 Introduction

1.1 Background and Motivation

1.2 Distributed Storage and Erasure Coding

1.2.1 Distributed Storage and Data Reliability

1.2.2 Erasure Coding over Finite Fields

1.2.3 XOR-Based Erasure Coding

1.2.4 Research Gaps and Opportunities for two-tone

1.3 Contributions

Chapter 2 Background

2.1 Principles of two-tone Encoding

2.2 Ceph EC Architecture and Integration Points

2.3 The Variable-Length Parity Problem

2.4 End-to-End Dataflow

Chapter 3 Encoder and Decoder Implementation and Optimization

3.1 Implementing the two-tone Plugin

3.1.1 Data Layout and Precomputation

3.1.2 Parameter Parsing and Initialization

3.1.3 Encoding

3.2 Optimizing Decoding Performance

3.2.1 From Iterative Peeling to a Fast Recovery Path

3.2.2 Tile-Based Cache-Friendly Decoding

3.2.3 Buffer Reuse and AVX2 SIMD XOR

Chapter 4 System Integration and Evaluation

4.1 Supporting Variable-Length Fragments

4.1.1 Plugin Interface and Actual-Length Awareness

4.1.2 Physical Storage and Block-Aligned Padding

4.1.3 Correcting Read Write and Recovery Paths

4.2 Adapting the OSD Layer

4.2.1 Encoding Path in ECUtil::encode

4.2.2 Normal Read Path in the First ECUtil::decode Overload

4.2.3 Recovery Path in the Second ECUtil::decode Overload and ECBackend

4.2.4 Adapting HashInfo

4.3 Experimental Setup and Methodology

4.3.1 Experimental Setup

4.3.2 Experiment Categories and Methods

4.4 Integration Validation

4.5 Core Encoding and Decoding Performance

4.5.1 Encoding Throughput for Large Chunks

4.5.2 Decoding Throughput for Large Chunks

4.6 System-Level Storage and Recovery Performance

4.6.1 rados bench Write and Read Performance

4.6.2 Failure Recovery Performance

Conclusion

References

Harbin Institute of Technology Undergraduate Thesis Declaration of Originality and Terms of Use

Acknowledgments

Chapter 1 Introduction

1.1 Background and Motivation

In the era of big data and cloud computing, data volumes are growing much faster than storage hardware capacity. IDC predicted that the global datasphere would exceed 175 ZB by 2025, with most of these data requiring persistent, highly reliable storage. Modern distributed storage systems must therefore minimize redundancy while preserving data durability and high-throughput access.

Replication is straightforward to implement and supports direct recovery, but it typically requires 2×-3× storage redundancy: each byte of useful data consumes 2-3 bytes of physical capacity. This overhead is costly for object stores at petabyte scale and beyond. Erasure coding provides a more flexible tradeoff between fault tolerance and capacity. A (k+m, k) code comprises k data chunks and m parity chunks, tolerates any m simultaneous chunk failures, and requires a storage overhead of only 1+m/k. For k=6, m=3, capacity utilization reaches 66.7%, compared with 33.3% for three-way replication.

Ceph is an open-source distributed storage system widely used in research and industry. Its object layer, `RADOS`, uses `CRUSH` for data placement and provides a scalable, mature implementation[1]. EC pools have become an important alternative to replicated pools for cold-data archives, large-object storage, and disaster recovery across data centers. Ceph's mainstream EC backends, Jerasure and ISA-L, implement Reed-Solomon codes through matrix operations over a finite field. Encoding, decoding, and recovery require many finite-field multiplications and matrix inversions, imposing substantial CPU costs. These costs can limit overall throughput on edge devices, cost-sensitive servers, and workloads with frequent recovery, such as data migration or OSD replacement. As a Shift-XOR storage code[3], two-tone XORs data chunks into parity chunks according to a predefined shift layout. It avoids finite-field multiplication entirely and offers the prospect of higher encoding efficiency and a simpler implementation.

Some two-tone parity chunks are inherently longer than data chunks. Integrating them into a large system such as Ceph, which assumes equal-sized fragments, creates problems at several layers. Post-encoding length assertions fail when parity chunks are too long; fixed-stride slicing in the read path misaligns fragment contents; recovery overestimates the stripe count when it derives that count from a longer parity fragment; and rollback and attribute-synchronization logic compute incorrect truncation offsets. These failures occur at different points in Ceph's data paths, with distinct triggers and remedies, and constitute the principal engineering obstacle to deploying two-tone in a real system. Mathematical analysis alone does not establish the practical value of a new code. Integrating it into a distributed storage system and evaluating object writes, reads, and parity-fragment recovery provide a stronger test. This thesis therefore uses Ceph to advance two-tone from an encoder that runs in unit tests to a production-grade EC plugin that runs reliably in real OSD data paths. We then evaluate system-level performance to bridge the gap between the theoretical code and its implementation.

1.2 Distributed Storage and Erasure Coding

1.2.1 Distributed Storage and Data Reliability

Distributed storage systems have evolved alongside the growth of the Internet. Early systems such as Google GFS[11] and Amazon Dynamo used replication to ensure reliability. As data volumes grew exponentially, the additional capacity cost became increasingly difficult to justify. Large cloud providers, including Facebook, Microsoft Azure, and Alibaba OSS, subsequently introduced erasure coding into their object stores. This approach targets 99.999999999% durability, or eleven nines, while increasing usable capacity from roughly 33% with three-way replication to 50%-75%[16].

Ceph is a prominent open-source distributed storage system in the cloud-native era. Its `CRUSH` consistent-hashing algorithm and pluggable EC framework make it an important platform for both research

and deployment[1][18]. An EC pool stripes each object into k data chunks and m parity chunks and distributes them across OSD nodes. It can tolerate m simultaneous node failures while supporting concurrent reads and writes. As clusters grow from tens to thousands of nodes, encoding, decoding, and recovery computation increasingly constrain aggregate throughput, motivating more efficient coding schemes[2].

1.2.2 Erasure Coding over Finite Fields

Reed-Solomon codes are among the most established erasure codes for distributed storage. They construct a generator matrix over a Galois Field (GF), allowing any k available fragments to reconstruct the original data through matrix inversion[4][5]. Existing implementations and related schemes include the following:

Jerasure provides Reed-Solomon Vandermonde matrices, Cauchy Reed-Solomon codes, and several SIMD optimization strategies. It was one of Ceph's earliest EC backends and remains the default configuration[17].

ISA-L (Intel Storage Acceleration Library) highly optimizes finite-field computation on Intel x86 platforms. It uses SSE4/AVX2/AVX-512 instructions to accelerate GF multiplication and substantially outperforms software-only implementations on high-performance servers. Error! Reference source not found.

CLAY codes use the minimum storage regenerating (MSR) framework to minimize recovery bandwidth and outperform conventional RS codes for coordinated recovery of multiple nodes[6].

Locally repairable codes (LRC) introduce local parity groups so that a single-node failure can be repaired by accessing only a small number of nearby fragments, reducing network traffic for routine single-disk failures[7].

These approaches offer mature theory, comprehensive recovery capabilities, and established ecosystems. Their main limitation is the cost of finite-field multiplication and matrix operations in encoding and decoding, which can become a bottleneck when CPU resources are limited.

1.2.3 XOR-Based Erasure Coding

Compared with finite-field codes, XOR-based erasure codes are simpler to implement, require lightweight operations, and parallelize readily[2][8]. Early examples include EVENODD and RDP for RAID-6[14][15], STAR for triple-failure tolerance, and the systematic XOR-based MDS framework introduced by Blömer et al.[8]. Each makes tradeoffs in parameter flexibility or fault tolerance.

Shift-XOR storage codes extend this approach by introducing positional shifts during encoding. They retain XOR-level computational complexity while allowing flexible fault-tolerance parameters. Prior decoding and repair schemes for Shift-XOR regenerating codes[9][10] provide the theoretical foundation. two-tone is a more recent example[3]. It constructs each parity chunk by XORing multiple data chunks at offsets specified by a two-tone shift layout. Its encoding and decoding paths avoid finite-field multiplication, naturally support SIMD vectorization, and accommodate any configuration with $m \leq k$. The shifts also introduce an inherent implementation property: parity chunk p must accommodate the shifted contributions of all data chunks. A contribution can extend beyond the original data length, so the effective parity length equals the data length plus the bytes required by the maximum shift. The resulting variable-length parity chunks create a new integration challenge.

1.2.4 Research Gaps and Opportunities for two-tone

Prior work largely addresses either coding theory and single-machine benchmarks or the performance of established Ceph plugins such as Jerasure, ISA-L, and CLAY[12]. Codes such as two-tone, which generate variable-length parity chunks, face more difficult issues in matching interface assumptions, identifying

slicing boundaries, computing recovery lengths, and implementing `encode()` and `decode()` in a real system. A complete, publicly available, and reproducible integration remains lacking. This gap between algorithmic feasibility and system usability motivates the work in this thesis.

1.3 Contributions

Our goal is to turn the theoretical two-tone code into an EC plugin that runs reliably in Ceph's real data paths. We address three technical problems and make the following contributions.

(1) A general encoder and decoder integrated as a plugin. We extend a fixed-parameter implementation to support any $m \leq k$. `TwotoneLayout` precomputes the shift matrix and actual parity-chunk lengths once during `prepare()`, eliminating repeated runtime derivation. We implement `ErasureCodeTwotone` within Ceph's `ErasureCodePlugin` framework and support dynamic parameter configuration through ceph osd erasure-code-profile set.

(2) Compatibility with variable-length fragments. two-tone's variable-length parity chunks conflict with the OSD layer's implicit equal-sized fragment assumption, causing write assertions, misaligned read slices, and incorrect recovery calculations. We add actual-length queries at the plugin interface, correct five instances of the assumption in the OSD layer, and combine block-aligned padding with logical trimming for physical storage. These changes enable rados put/get/bench and the complete OSD failure-recovery path.

(3) Successive optimizations of the decoding path. We implement a single-fragment fast recovery path that bypasses the peeling state machine, tile-based cache-friendly column-first loops, preallocated temporary buffers, and an AVX2 SIMD XOR kernel. Together, these changes increase core decoding throughput from approximately 0.1 GB/s in the initial implementation to 13.23-17.89 GB/s for large chunks, while consistently outperforming Jerasure RS in system-level recovery tests.

In the algebraic framework of two-tone codes, striping the original data stream is the starting point for Shift-XOR computation. Conventional erasure codes divide data at byte boundaries into fixed matrix elements; two-tone further represents each data chunk as a sequence of bit groups or fixed-width words. Constructing this variable-length algebraic stream balances computational granularity against memory bandwidth across the storage hierarchy. Our plugin explicitly defines 128-bit logical block boundaries for these sequences. This design allows shifts to be prealigned to integer multiples of memory-page or cache-line boundaries. Incorporating hardware considerations into the striping definition supports subsequent SIMD acceleration and helps explain the high throughput achieved for large, contiguous I/O.

Chapter 2 Background

2.1 Principles of two-tone Encoding

two-tone uses a two-tone shift structure: each parity chunk P combines data chunks D by XORing them at different offsets. A cell is the smallest logical unit and contains 16 bytes in this implementation. The value at position t in a parity chunk is:

$$P_p[t] = XOR\{D_d[t - s_{p,d}] | d \in [0, k), t - s_{p,d} \in [0, C)\}$$

Here, $s_{p,d}$ is the shift between parity chunk p and data chunk d, measured in cells, and C is the number of cells in a data chunk. The function `twotone_shift` (p, d, k, m) computes the shift as follows:

$s_{p,d} = p \cdot k + d$ (an integer staircase progression that assigns a unique value to each (p,d) pair)

This definition assigns a distinct offset to every (p, d) pair and satisfies the condition for unique decodability of two-tone codes.

When data chunk D contributes to parity chunk P, its starting position is shifted by $s_{p,d}$ cells. The final $s_{p,d}$ cells therefore contribute to the tail of P, making its effective length greater than that of D:

$$len(P_p) = C + max_shift[p] \times BYTE_PERCELL$$

Here, $max_shift[p] = max\{s_{p,d} | d \in [0, k)\}$, and BYTE_PERCELL = 16 is the cell size used for AVX2 alignment.

The Shift-XOR mechanism has a distinctive computational dataflow when viewed as a joint design of mathematics and computer architecture. Predefined integer staircase shifts replace conventional finite-field matrix multiplication with linear memory offsets and word-aligned XOR operations. Modern superscalar processors execute XOR instructions at high throughput and readily pipeline them. Trading a small increase in fragment length for simpler computation underpins the practical performance potential of two-tone codes.

Figure 2-1 illustrates the Shift-XOR encoding principle.

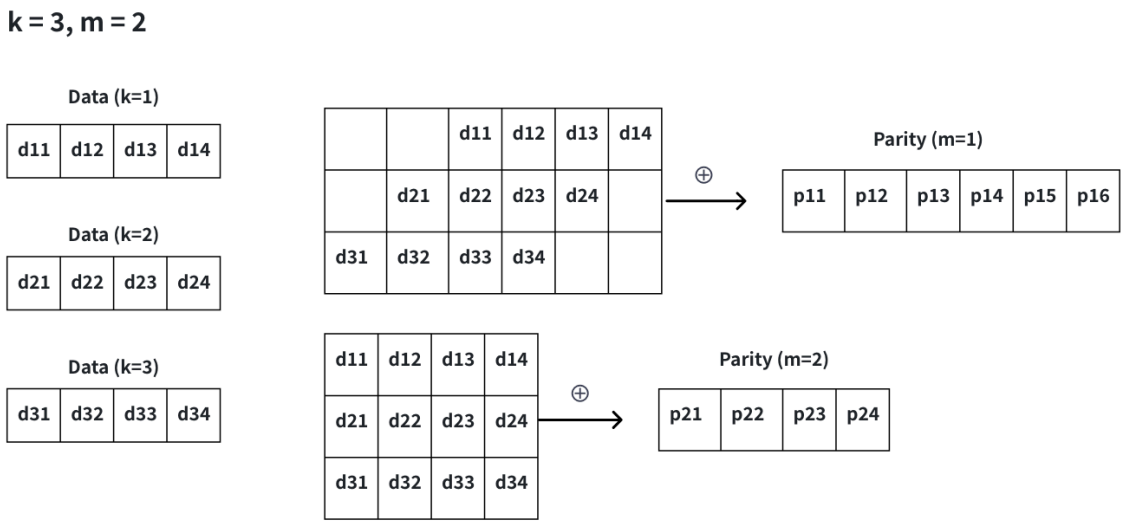


Figure 2-1 Principles of two-tone encoding

2.2 Ceph EC Architecture and Integration Points

Ceph organizes storage into three layers: client interfaces (librados/RBD/CephFS/RGW), the object layer (`RADOS`/OSD), and the storage-media layer (`BlueStore`/FileStore). The core EC logic resides in the OSD layer and spans the following components.

`ErasureCodeInterface` defines the standard interface implemented by every EC plugin, including `encode_chunks()`, `decode_chunks()`, and `get_chunk_size()`.

`ErasureCodePlugin` provides plugin loading and factory registration through dynamically linked libraries (.so) loaded at runtime.

`ECUtil` provides utilities for striping, slicing, and reassembling bufferlist objects and serves as the OSD layer's common entry point to EC plugins.

`ECBackend` manages reads, writes, and the recovery state machine for EC objects. It distributes fragment-write requests across OSDs and coordinates recovery pushes (`PushOp`).

Ceph's EC plugin framework embeds a strong equal-sized fragment assumption in its core ErasureCode abstraction. Understanding the framework's limitations in tracking stripe boundaries is therefore essential both to assessing the feasibility of two-tone integration and to adapting the underlying OSD I/O paths for variable-length fragments.

Our two-tone plugin integrates with `ErasureCodePlugin` and associates with a storage pool through a dynamically configured EC Profile. Figure 2-2 shows the integration point and the scope of the changes within Ceph.

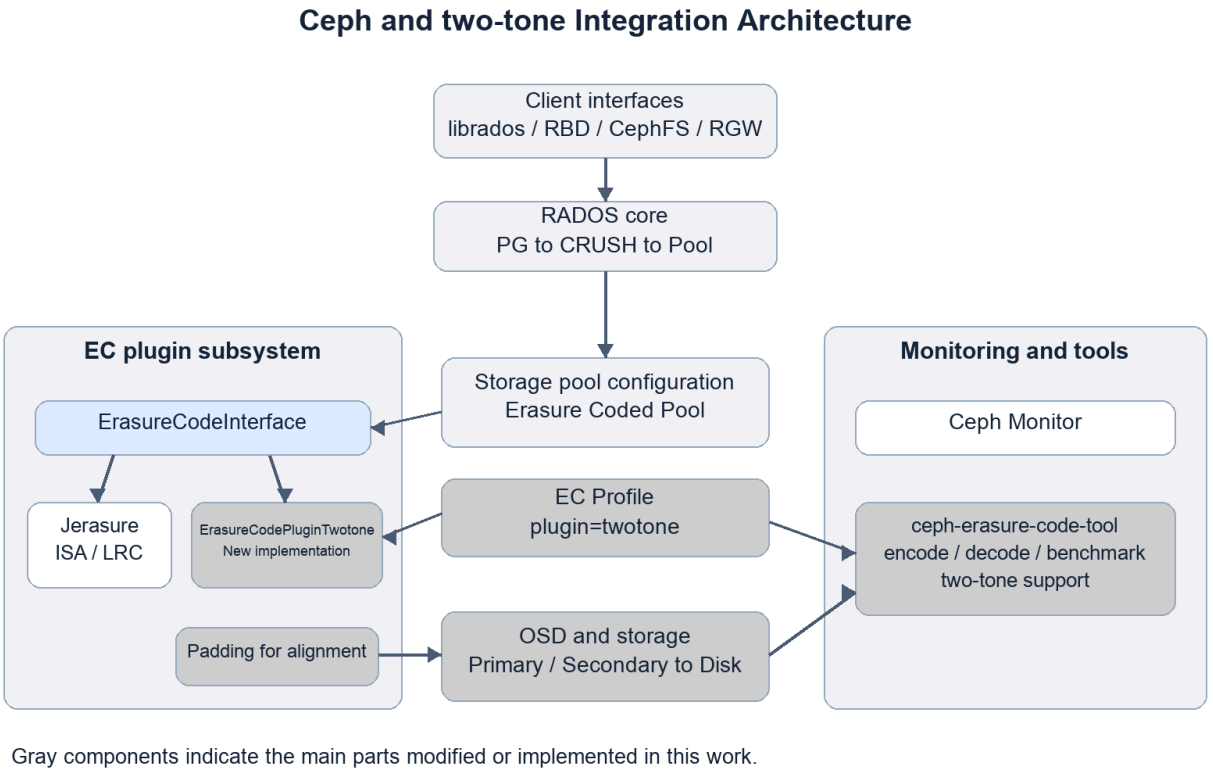


Figure 2-2 Ceph architecture and two-tone plugin integration

Ceph implements erasure coding over logical stripes.

Figure 2-3 illustrates encoding and decoding with striping for k=3, m=2.

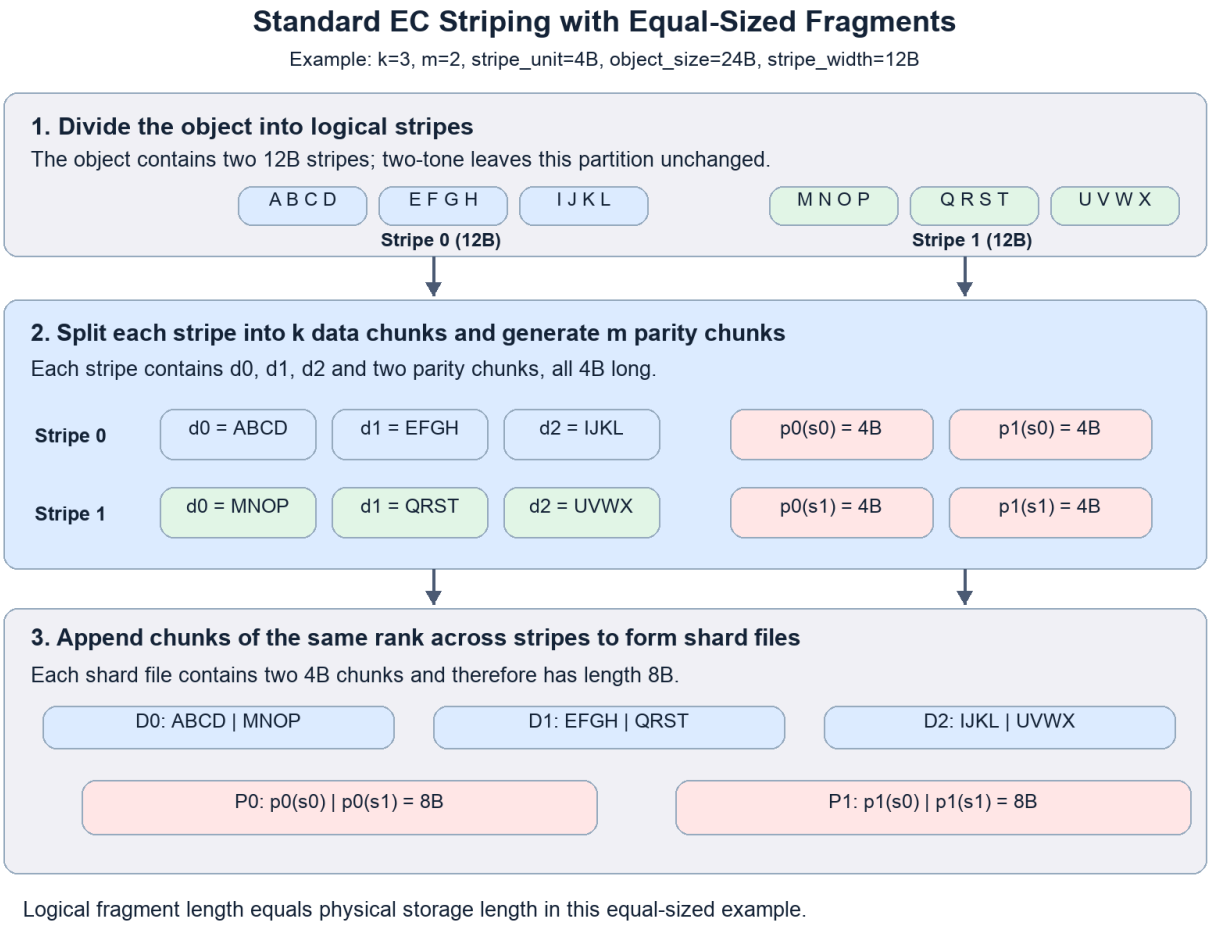


Figure 2-3 Standard striping logic for two-tone

In Ceph's standard striping implementation, conventional EC first divides a 24B object into two logical stripes with `stripe_width` = 12B. Each stripe then yields three equal-sized data chunks, d0/d1/d2, with `stripe_unit` = 4B, and two equal-sized parity chunks, p0/p1. Chunks of the same rank are appended across stripes, producing five shard files of 8B each. Logical fragment length equals physical storage length, and the equal-sized fragment assumption holds throughout the data path.

Figure 2-4 further illustrates the relationship between logical fragment length and physical storage length.

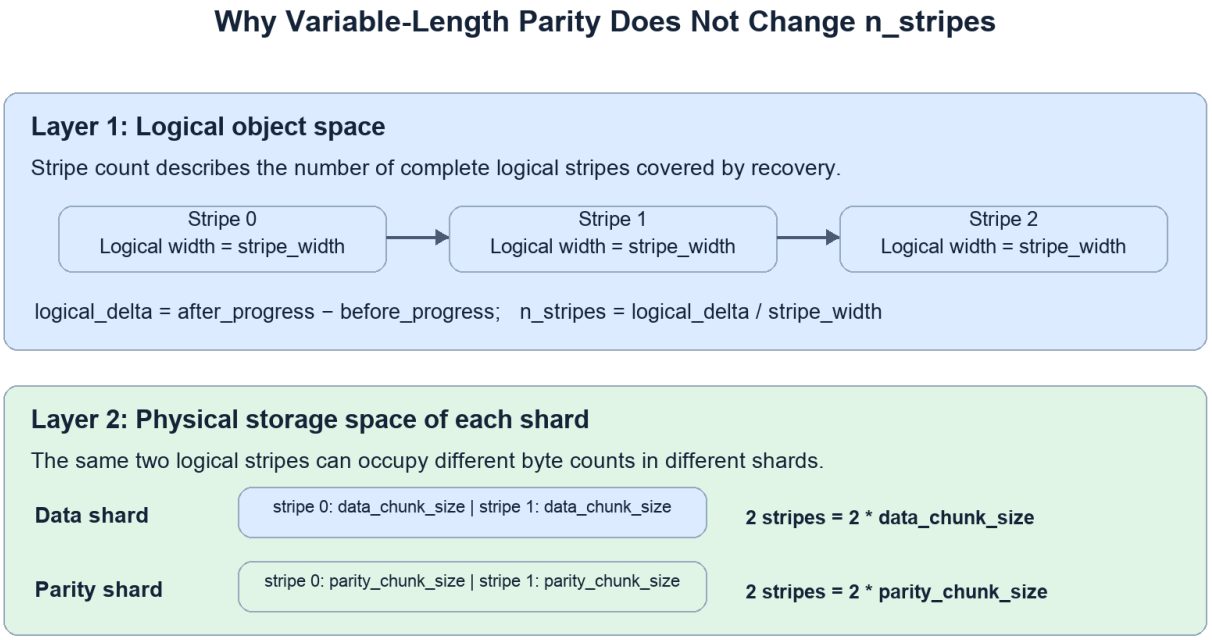


Figure 2-4 Logical fragment length and physical storage length

2.3 The Variable-Length Parity Problem

When we first integrated two-tone into a Ceph cluster, the main obstacle was a mismatch between its fragment lengths and the system's default assumption that all data and parity chunks have equal length. Debugging identified five instances of this assumption at different stages of the data path, each producing a different failure.

(1) Fragment-length assertions in `ECUtil::encode()`

After encoding, `ECUtil::encode()` asserts that every output fragment has length `data_chunk_size`:

`ceph_assert(bl.length() == chunk_size);` // Same assertion for every shard

A two-tone parity fragment has length `data_chunk_size + max_shift[p] * 16`, which immediately triggers the assertion and crashes the process.

(2) Fixed-stride slicing in the first overload of `ECUtil::decode()`

The normal-read overload of `decode()` slices a concatenated bufferlist at a fixed stride of `data_chunk_size`. When the concatenated result includes variable-length parity fragments, the slice boundaries shift, misaligning the contents of subsequent fragments.

(3) Incorrect `n_stripes` inference in the second overload of `ECUtil::decode()` for recovery

The recovery overload of `decode()` infers the stripe count `n_stripes` from the length of an available fragment. If that fragment is a longer parity fragment, it overestimates `n_stripes` and consequently computes incorrect slicing strides and decoding input ranges.

(4) Range calculations in `ECBackend::handle_recovery_read_complete()`

The recovery path computes fragment-data ranges using `aligned_logical_offset_to_chunk_offset(delta)`, which returns (delta / `stripe_width`) *

`data_chunk_size`. This expression is correct for data fragments but underestimates the upper bound for parity fragments. The resulting `data_included` byte range in the recovery `PushOp` is incorrect and causes the OSD to crash.

(5) Equal-length assertions in `ECUtil::HashInfo::append()`

When appending fragment CRCs, `HashInfo` asserts that all fragments have equal length to check stripe integrity. Variable-length two-tone parity chunks also violate this assertion.

Figures 2-5 and 2-6 compare the `RADOS` operation paths before and after adaptation.

Before adaptation, a typical rados put crashes in unmodified Ceph because the encoder generates parity fragments longer than `data_chunk_size`. The assertion `ceph_assert(bl.length() == chunk_size)` in `ECUtil::encode()` fails before the data persist to disk. After the five corrections, the same operation completes normally. `ECUtil::encode()` queries actual lengths by fragment type, allowing the assertion to pass. The write, read, and recovery paths correctly handle variable-length parity fragments and complete persistence and reconstruction. This comparison shows that the five corrections cover the equal-sized fragment assumptions identified in the Ceph OSD layer and remove the remaining engineering obstacle to running two-tone in a real cluster.

Before adaptation (assertion failure):

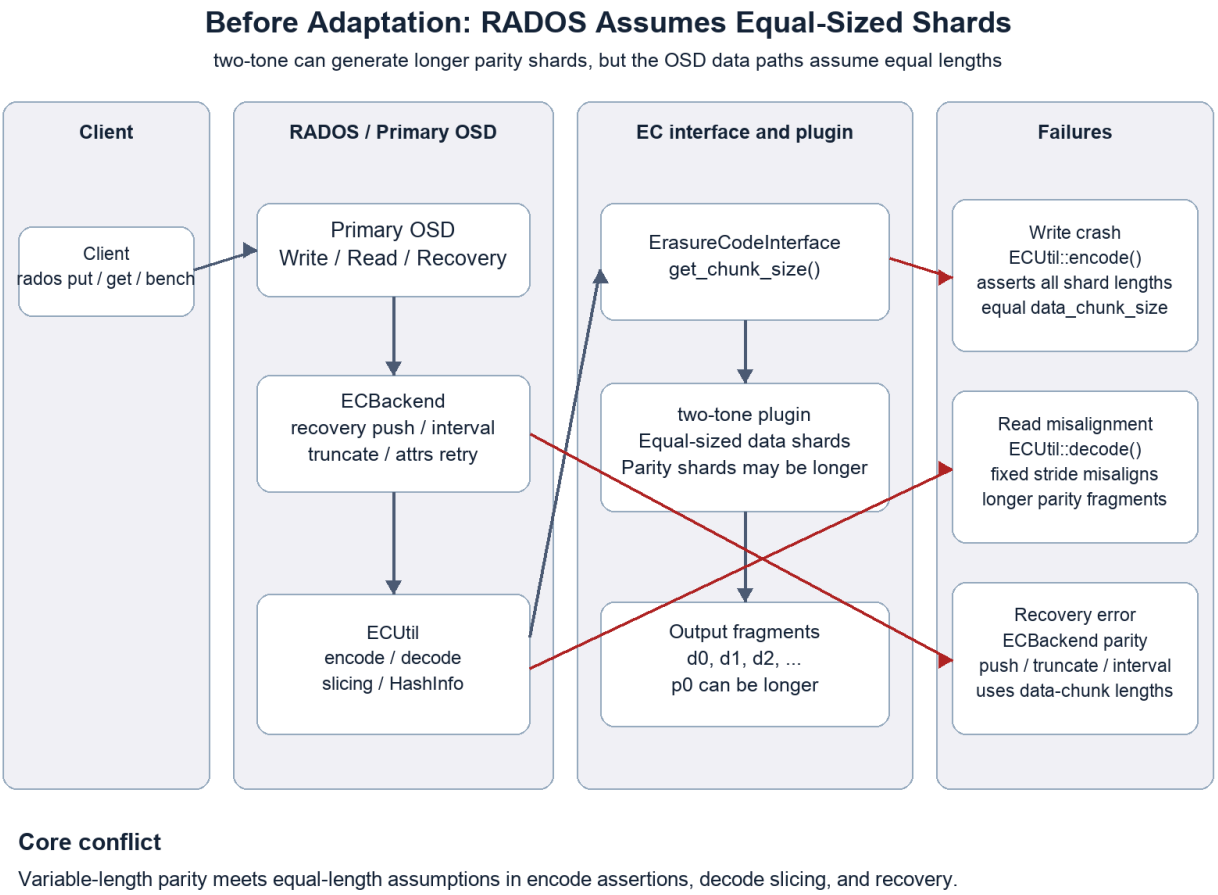


Figure 2-5 `RADOS` operation paths before two-tone adaptation

After adaptation (normal operation):

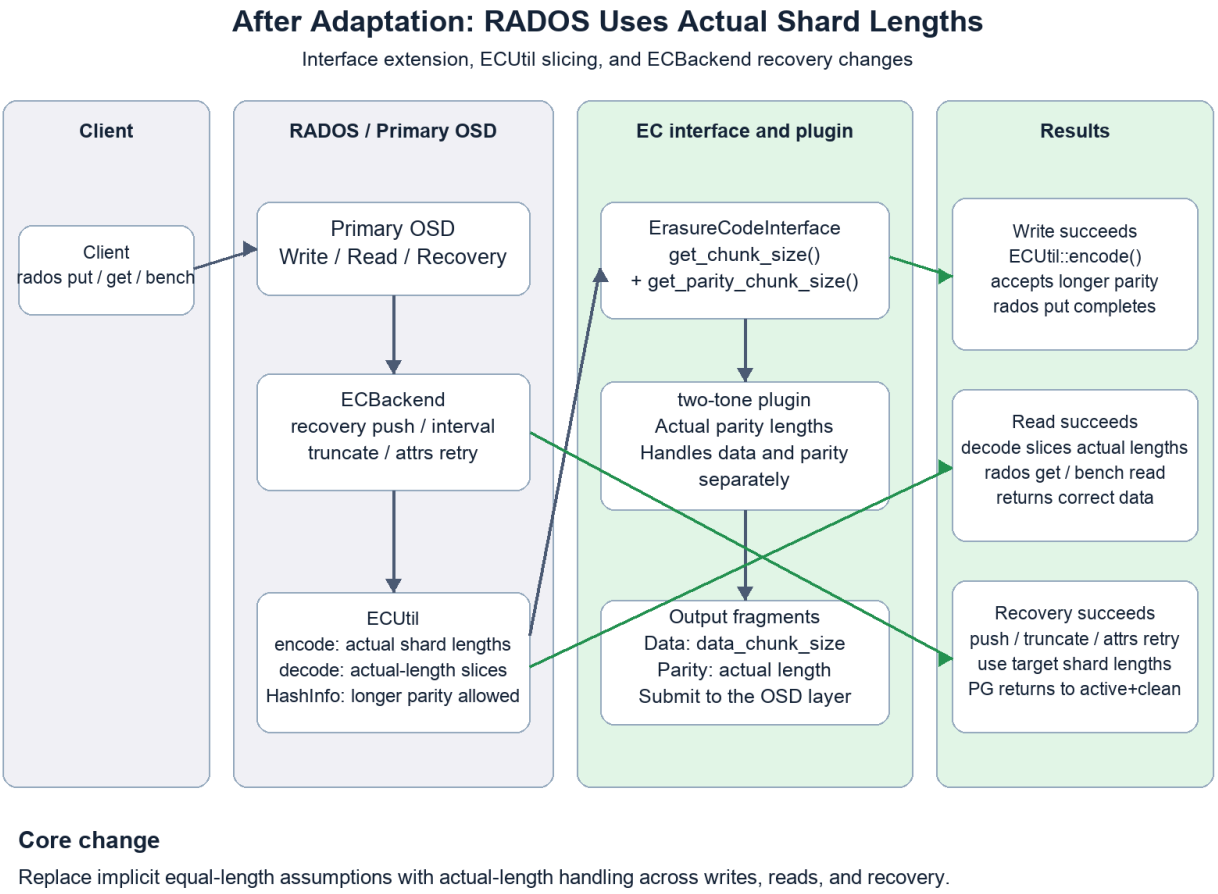


Figure 2-6 RADOS operation paths after two-tone adaptation

2.4 End-to-End Dataflow

Our design preserves Ceph's existing dataflow wherever possible and makes the necessary corrections where its length assumptions fail. It does not introduce a separate data path for two-tone.

Object writes proceed as follows: the client issues a write through librados → `RADOS` maps the object to a PG (Placement Group) → the primary OSD calls `ECUtil::encode()` to divide the object into k data chunks → `ErasureCodeTwotone::encode_chunks()` generates m variable-length parity chunks → padding extends all fragments to a common physical length → `BlueStore` persists them to disk.

We introduce a physical padding layer before persistence to reconcile variable-length code chunks with Ceph's stripe-alignment constraints. The system queries each fragment's actual logical length through the plugin, finds the maximum, and transparently zero-pads shorter data and parity fragments at the tail of their memory buffers. This preserves the original two-tone algebraic stream while meeting `BlueStore`'s requirements for block-aligned writes. Physical I/O can then be submitted as standard, equal-sized concurrent blocks and safely persisted.

Object reads proceed as follows: the primary OSD obtains a set of available fragments sufficient for decoding → `ECUtil::decode()` slices them using their actual lengths → `ErasureCodeTwotone::decode_chunks()` reconstructs missing fragments → the original object is reassembled → the object is returned to the client.

The primary OSD must collect any k available fragment buffers, potentially including variable-length parity chunks. To present correct boundaries to the decoder, we implement actual-length-aware slicing in `ECUtil::decode()`. Using metadata, the system computes each shard's logical boundary and applies logical

trimming to remove physical alignment zeros from its tail. It then passes correctly aligned streams to `decode_chunks()`, avoiding out-of-bounds accesses and decoding failures during degraded reads.

Failure recovery proceeds as follows: Ceph Monitor detects an OSD failure and the PG becomes degraded → `ECBackend` reads enough fragments from surviving OSDs → the decoder reconstructs missing fragments → `PushOp` is constructed using actual fragment lengths → the fragments are sent to the target OSD and persist to disk → the PG returns to active+clean.

Completing all three paths is the key criterion for establishing that the plugin is usable in a real system.

Chapter 3 Encoder and Decoder Implementation and Optimization

3.1 Implementing the two-tone Plugin

We implement the complete `ErasureCodeTwotone` class within Ceph's `ErasureCodePlugin` framework. It comprises the following modules.

The parameter-parsing and initialization module reads k and m from the EC Profile, validates them, and calls `build_layout()` to construct the `TwotoneLayout` tables required at runtime. It ensures that the plugin initializes correctly when a pool is created. The coding module provides `encode_chunks()` and `decode_chunks()`. The former constructs variable-length parity chunks using Shift-XOR, while the latter implements a peeling-based single-fragment fast recovery path and ultimately uses an AVX2 SIMD kernel. Its performance determines how well two-tone competes under real workloads. The factory-registration module implements `ErasureCodePluginTwotone` so that Ceph can load the plugin dynamically at runtime and associate it with plugin=twotone in the EC Profile.

3.1.1 Data Layout and Precomputation

To support any configuration with m≤k, we replace the hard-coded k=3, m=3 implementation with the following `TwotoneLayout` structure:

```
struct TwotoneLayout {
    int k, m;
    std::vector<int> max_shift;
    std::vector<int> shift;
};
```

`TwotoneLayout` contains two key fields. `max_shift` is a vector of length m; `max_shift[p]` records the maximum shift across all data chunks for parity chunk p. This value determines the parity chunk's actual storage length and underlies variable-length fragment compatibility. shift is a flattened array of length m×k that stores the two-dimensional shift table in row-major order. The entry shift[p*k+d] gives the shift $s_{p,d}$ between parity chunk p and data chunk d. Encoding and decoding loops access the array directly, avoiding two-dimensional index conversion.

The layout-construction function `build_layout(k, m)` traverses all (p, d) pairs, calls `twotone_shift(p, d, k, m)` to populate shift, and updates `max_shift[p]`, as shown in Algorithm 3-1.

Algorithm: Construct the two-tone shift layout (build_layout)
Input: Number of data chunks k; number of parity chunks m
Output: Initialized layout structure layout
1: layout.k ← k, layout.m ← m
2: Initialize layout.max_shift as a zero-filled vector of length m
3: Allocate m * k entries for layout.shift
4: For each parity-chunk index p from 0 to m - 1:
5: For each data-chunk index d from 0 to k - 1:
6: s ← twotone_shift(p, d, k, m)
7: layout.shift[p * k + d] ← s
8: layout.max_shift[p] ← max(layout.max_shift[p], s)
9: Return layout

Algorithm 3-1 Constructing the two-tone shift layout

`TwotoneLayout` is built once during `init()`. Subsequent calls to `encode_chunks()`, `decode_chunks()`, `get_parity_chunk_size()`, and OSD-layer functions use the precomputed results directly. Avoiding repeated derivation of offsets on the critical path reduces runtime control overhead and improves maintainability.

3.1.2 Parameter Parsing and Initialization

The `parse()` method reads k and m from the EC Profile:

```
int ErasureCodeTwotone::parse(const ErasureCodeProfile &profile) {
    int err = ErasureCode::parse(profile);
    k = profile.count("k") ? stoi(profile.at("k")) : 3;
    m = profile.count("m") ? stoi(profile.at("m")) : 2;
    return err;
}
```

The `init()` method checks $m \leq k$, as required for convergence of the two-tone decoder, and calls `build_layout(k, m)` to initialize the layout:

```
int ErasureCodeTwotone::init(ErasureCodeProfile &profile, ...) {
    int err = parse(profile);
    if (m > k) { return -EINVAL; }
    layout = build_layout(k, m);
    return err;
}
```

3.1.3 Encoding

`encode_chunks()` places the data-chunk loop outside the parity-chunk loop to reduce repeated reads of the same data chunk, as shown in Algorithm 3-2.

Algorithm: Shift-XOR encoding of variable-length two-tone parity chunks (encode_chunks)
Input: Data-chunk set data; variable-length parameter layout layout
Output: Generated variable-length parity-chunk set coding
1: For each parity-chunk index p from 0 to m - 1:
2: Allocate coding[p] at its actual logical length and zero the buffer
3: For each data-chunk index d from 0 to k - 1:
4: src_ptr ← starting address of data[d]
5: For each parity-chunk index p from 0 to m - 1:
6: shift_bytes ← layout.shift[p * k + d] * bytes per cell
7: dst_ptr ← starting address of coding[p] + shift_bytes
8: Call xor_buf(dst_ptr, src_ptr, total bytes in the data chunk)
9: Return success

Algorithm 3-2 Shift-XOR encoding of variable-length parity chunks

The implementation first allocates every parity chunk at its actual length `parity_chunk_size(p)` and zeroes it. It then iterates over data chunks. For each chunk d, it obtains the source pointer src and traverses all parity chunks p in the inner loop. The precomputed shift table supplies `shift_bytes`; dst points to the corresponding offset in the parity chunk; and `xor_buf()` XORs the entire data chunk into that destination in place. Iterating over data chunks before parity chunks ensures that each data chunk is read only once, avoids repeated loads associated with the reverse traversal, and improves last-level cache (LLC) locality. When AVX2 is available, `xor_buf()` uses 256-bit vector XOR operations.

Reading each data chunk only once reduces pressure on the LLC.

3.2 Optimizing Decoding Performance

The initial two-tone implementation was functionally correct but decoded very slowly. Debug logs showed that `decode_chunks()` achieved only about 0.1 GB/s, substantially below Jerasure. We optimized it in three stages.

3.2.1 From Iterative Peeling to a Fast Recovery Path

Root cause. The initial `decode_chunks()` used a cell-by-cell peeling process, similar to LDPC decoding. It maintained vector<vector<bool>> to track whether each (chunk, cell) position was known and repeatedly searched for solvable equations in an outer `while(progress)` loop. This approach incurred substantial control overhead and branching, and the bit-packed access pattern of vector<bool> prevented vectorization.

Optimization. For the common case of a single missing fragment, including the rados bench case with one lost fragment, we implement a fast path that directly reverses encoding:

1. Copy the corresponding parity chunk into the destination buffer.

2. XOR all known data chunks using the same shift relationships.

3. The remaining data form the missing chunk.

This path bypasses state tracking entirely. Algorithm 3-3 shows the core loop.

Algorithm: Fast inverse-XOR decoding for a single missing fragment
Input: Missing data-chunk index d_miss; available parity chunk parity[p]; surviving data chunks data; layout layout
Output: Reconstructed data chunk dst
1: Copy the first data_chunk_size bytes of parity[p] into dst
2: For each data-chunk index d from 0 to k - 1:
3: If d equals d_miss:
4: Continue
5: shift_bytes ← layout.shift[p * k + d] * bytes per cell
6: /* XOR the leading shifted region */
7: xor_buf(dst, data[d] + shift_bytes, data_chunk_size - shift_bytes)
8: /* XOR the wrap-around tail region */
9: xor_buf(dst + data_chunk_size - shift_bytes, data[d], shift_bytes)
10: Return success

Algorithm 3-3 Fast decoding for a single missing fragment

The loop contains only XOR and memcpy operations and can be vectorized automatically by the compiler.

Result. For k=4, m=2, decoding throughput rises from 0.1 GB/s to 0.6 GB/s, approximately a sixfold improvement.

3.2.2 Tile-Based Cache-Friendly Decoding

Root cause. Although the fast path simplifies the algorithm, repeatedly reading and writing entire megabyte-scale buffers still makes memory bandwidth a bottleneck. In particular, parity data exceed the L1 cache and are repeatedly fetched from memory.

Optimization. We introduce tiled, column-first loops. Each iteration processes a block of TILE_SIZE, typically 16-64 KB to match the L1 cache, and completes all XOR operations involving that block before proceeding:

```
for tile_start in range(0, data_chunk_size, TILE_SIZE):
    load parity[tile_start:tile_start+TILE_SIZE] into scratch_buf
    for each known data chunk d:
        xor scratch_buf with data[d][...] (shifted accordingly)
    write scratch_buf to dst
```

Each parity tile is loaded only once, and XOR operations over multiple data chunks complete within the cache. This substantially reduces traffic between the LLC and main memory.

3.2.3 Buffer Reuse and AVX2 SIMD XOR

Buffer preallocation. We make the scratch buffer a member variable and allocate it once during `init()`, avoiding allocation overhead on every decoding call.

AVX2 XOR kernel. The core XOR operation uses 256-bit AVX2 registers to process 32 bytes per iteration:

```
void xor_buf_avx2(uint8_t *dst, const uint8_t *src, size_t len) {
    size_t i = 0;
    for (; i + 32 <= len; i += 32) {
        __m256i a = _mm256_loadu_si256((__m256i*)(dst + i));
        __m256i b = _mm256_loadu_si256((__m256i*)(src + i));
        _mm256_storeu_si256((__m256i*)(dst + i), _mm256_xor_si256(a, b));
    }
    for (; i < len; i++) dst[i] ^= src[i];  // Handle the tail
}
```

Because two-tone uses only XOR operations, AVX2 provides substantial acceleration. For k=4, m=2, enabling AVX2 increases core encoding throughput by approximately 56% relative to the implementation without SIMD.

Table 3-1 summarizes the optimization stages.

Table 3-1 Decoding throughput by optimization stage (k=4, m=2; GB/s)

Optimization stage	Decoding throughput	Speedup over initial version
Initial version (Peeling, Vector<>)	0.1	1×
Fast path + flat buffers	0.6	6×
Tile-based cache optimization	~2.0	~20×
AVX2 SIMD XOR (large-chunk test)	13.2	~132×

Chapter 4 System Integration and Evaluation

4.1 Supporting Variable-Length Fragments

To resolve the five conflicts, we introduce a three-layer compatibility mechanism: actual-length awareness, block-aligned padding, and logical trimming.

4.1.1 Plugin Interface and Actual-Length Awareness

We expose actual fragment-length queries as a virtual function in `ErasureCodeInterface`. The OSD layer calls this common interface, and each plugin supplies its own implementation. This extends the interface without requiring changes to existing plugins.

We add the following virtual function to `ErasureCodeInterface`. The `parity_idx` argument identifies the parity chunk whose length is queried, enabling the caller to obtain the appropriate `chunk_size` for data and parity chunks:

```
virtual unsigned int get_parity_chunk_size(
    unsigned int object_size, int parity_idx = 0) const {
    return get_chunk_size(object_size);
}
```

Existing plugins such as Jerasure, ISA-L, LRC, and CLAY require no changes: their parity chunks already have the same length as data chunks, so the default implementation is correct. two-tone overrides the method to return the actual length:

```
unsigned int ErasureCodeTwotone::get_parity_chunk_size(
    unsigned int object_size, int parity_idx) const {
    return get_chunk_size(object_size)
           + layout.max_shift[parity_idx] * BYTE_PERCELL;
}
```

The OSD layer can therefore query any fragment's actual size through a common interface without checking the plugin type.

Figure 4-1 shows the two-tone dataflow after adding length awareness. Logical stripe boundaries and data-chunk partitioning remain unchanged; parity-chunk lengths differ. For example, p0 has logical length $4B + \Delta$ within each stripe, where $\Delta = \texttt{max_shift[0]} \times 16B$, whereas p1 remains 4B long. After chunks are appended across stripes, the P0 shard file is longer than the data shard files, while P1 retains the same length. The stripe count `n_stripes` does not change. Instead, each parity shard may contain a different number of bytes per stripe. This distinction is the common source of the failures caused by Ceph's equal-sized fragment assumption.

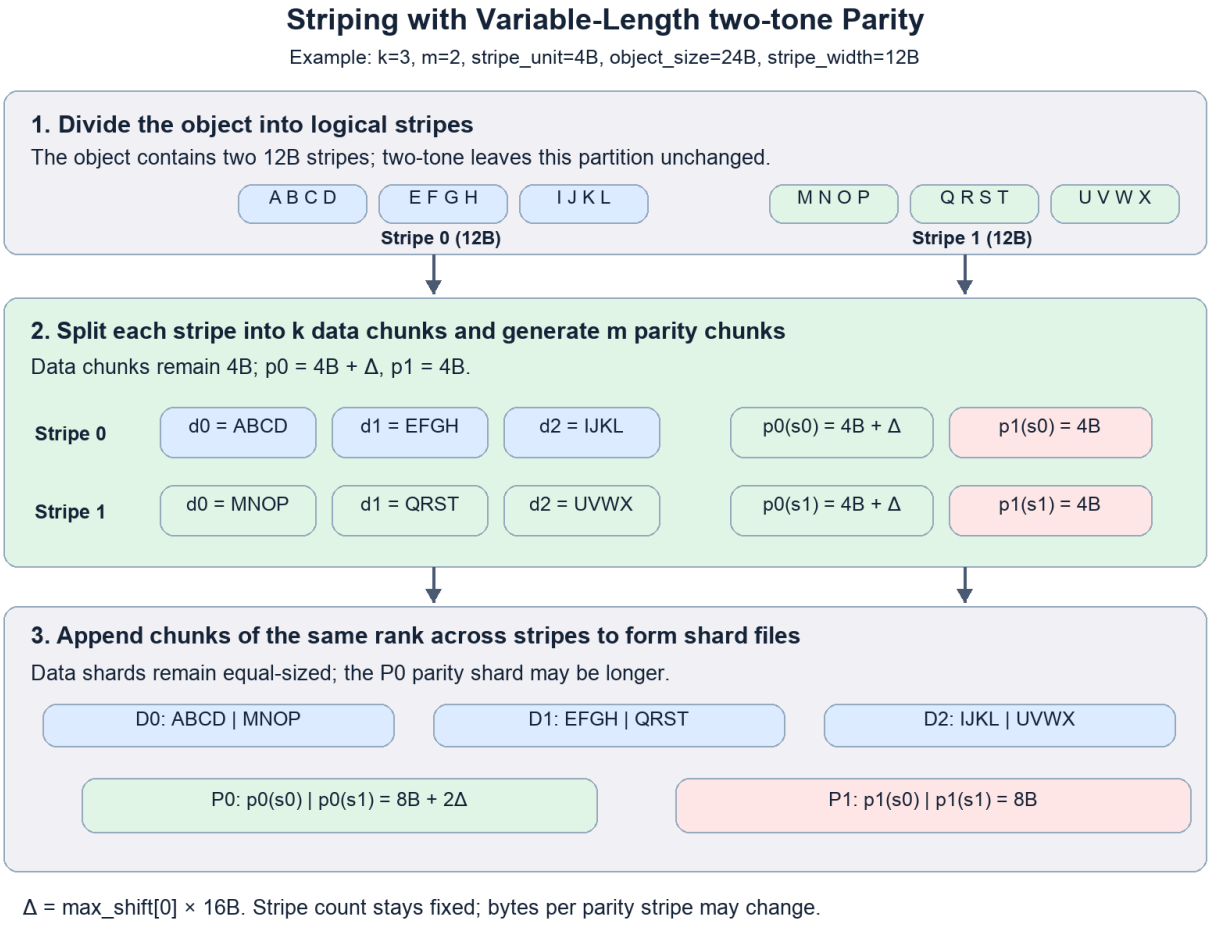


Figure 4-1 Revised striping for two-tone

4.1.2 Physical Storage and Block-Aligned Padding

Ceph requires fragments to be aligned to `stripe_unit` when they persist to disk. For variable-length parity fragments, we use the maximum fragment length within a stripe and pad all fragments to:

$$C_{store} = \lceil \frac{max(data_chunk_size, parity_chunk_size_0, \ldots, parity_chunk_size_{m-1})}{A} \rceil \times A$$

Here, A is the alignment granularity, typically 4096 bytes. Shorter data and parity fragments are zero-padded to C_{store} before physical writes. During reads and recovery, `get_parity_chunk_size()` supplies the logical length used to trim the invalid tail. Figure 2-4 illustrates the corresponding logical-to-physical mapping.

4.1.3 Correcting Read Write and Recovery Paths

Guided by the debugging results, we correct each of the five conflicts:

`ECUtil::encode()` — Replace the common length assertion with fragment-specific checks: data fragments must have length `data_chunk_size`, and parity fragments must have length `get_parity_chunk_size(stripe_width, shard_id - k)`.

`ECUtil::decode()` (first overload) — Infer `n_stripes` from the first available data fragment rather than an arbitrary fragment, and query each fragment's actual length to determine its slicing stride.

`ECUtil::decode()` (second overload, recovery path) — Correct the logic that infers `chunks_count` from an arbitrary lowest-index shard. Use `get_parity_chunk_size()` for parity-fragment strides instead of slicing every fragment at `data_chunk_size`.

`ECBackend::handle_recovery_read_complete()` — When the recovery target is a parity fragment, compute the upper bound of its data range with `get_parity_chunk_size()` instead of `aligned_logical_offset_to_chunk_offset()`. Also correct the truncation-offset calculation in `rollback_append()`.

`HashInfo::append()` — Allow parity fragments to have different lengths in the length assertion. CRC computation remains unchanged because it already operates independently on each fragment.

4.2 Adapting the OSD Layer

Adapting the OSD layer is the most technically demanding part of the project and offers the greatest engineering value. The challenge is not the amount of code. Once a length assumption fails, it can manifest as an assertion failure, object corruption, or failed recovery. Each crash points to a different stack location, requiring separate root-cause analysis.

4.2.1 Encoding Path in `ECUtil::encode`

After encoding, `ECUtil::encode()` checks each output fragment's length to verify that the plugin behaves as expected. Originally, it used the same expected `chunk_size`, namely `data_chunk_size`, for every fragment. A longer two-tone parity chunk immediately triggered the assertion and terminated the process. The assertion implicitly assumed equal-sized fragments, which two-tone does not provide. We instead query the expected length by fragment index. Data fragments with indices below k still use `data_chunk_size`, while parity fragments use `get_parity_chunk_size(stripe_width, shard_id - k)`. This preserves the assertion's ability to detect invalid plugin output while allowing valid variable-length parity chunks.

Before:

```
for (auto &p : out) {
    ceph_assert(p.second.length() == chunk_size);  // Same assertion for every fragment
}
```

After:

```
for (auto &[shard_id, bl] : out) {
    unsigned expected = (shard_id < k)
        ? data_chunk_size
        : ec_impl->get_parity_chunk_size(stripe_width, shard_id - k);
    ceph_assert(bl.length() == expected);
}
```

This change is transparent to existing plugins such as Jerasure and ISA-L. Their parity chunks have the same length as data chunks, so the assertion behaves exactly as before.

4.2.2 Normal Read Path in the First `ECUtil::decode` Overload

Before: slicing used a common stride of `data_chunk_size`, misaligning variable-length parity fragments.

After: `n_stripes` is derived from the length of the first data fragment (`shard_id` < k), avoiding distortion from longer parity fragments. The slicing loop computes `per_shard_size` separately for each fragment, as shown in Algorithm 4-1.

Algorithm: Actual-length-aware slicing of logical buffers in the normal read path
Input: Map of received raw buffers in; underlying EC interface instance ec_impl
Output: Map of sliced and logically trimmed buffers out
1: /* Infer the stripe count from the first native data fragment to avoid distortion from longer parity fragments */
2: n_stripes ← physical length of the first available data fragment / data_chunk_size
3: stripe_width ← n_stripes * data_chunk_size
4: For each [shard_id, bl] in the input fragment map in:
5: If shard_id < k:
6: per_shard_size ← data_chunk_size
7: Else: /* Parity fragment */
8: /* Query this variable-length parity chunk's actual length per stripe */
9: per_shard_size ← ec_impl.get_parity_chunk_size(stripe_width, shard_id - k)
10: Slice per_shard_size bytes from the start of bl and append them to out[shard_id]

Algorithm 4-1 Actual-length-aware slicing of logical buffers

4.2.3 Recovery Path in the Second `ECUtil`::decode Overload and `ECBackend`

Recovery failures are subtler than normal-read failures. The recovery target may itself be a parity fragment. Computing its range from the data-fragment length can cause a crash after reconstruction. We make the following corrections:

Second overload of `ECUtil`::decode: initialize `chunks_count` by preferentially inferring it from a data fragment, and use the appropriate `per_shard_size` for each output shard.

`ECBackend`::`handle_recovery_read_complete()`: when recovering a parity fragment, use `get_parity_chunk_size()` to compute the upper bound of `data_included`:

```
unsigned shard_data_size = (shard_id < k)
    ? sinfo.get_chunk_size()
    : ec_impl->get_parity_chunk_size(recovery_size, shard_id - k);
```

`ECBackend`::`rollback_append()`: correct the rollback truncation offset by using the actual parity-fragment length instead of `data_chunk_size`.

4.2.4 Adapting `HashInfo`

We restrict the equal-length assertion in `HashInfo`::`append()` to data fragments within a stripe, allowing parity fragments to be longer. This does not affect CRC tracking:

```
// Before: assert all shards equal
// After: assert equal lengths only for data fragments; exclude parity fragments
if (!shard_id_is_parity(id)) {
    ceph_assert(bl.length() == size_to_append);
}
```

4.3 Experimental Setup and Methodology

4.3.1 Experimental Setup

We conduct the experiments with the following hardware and software:

Operating system and hardware: Ubuntu 24.04 Linux on an x86_64 server with AVX2 support.

Ceph version: a development version based on the Ceph squid branch, with our `ErasureCodeTwotone` plugin integrated.

Storage backend: `BlueStore` with its default configuration.

EC plugins compared: two-tone (our implementation), Jerasure Reed-Solomon Vandermonde, and ISA-L Reed-Solomon.

4.3.2 Experiment Categories and Methods

Category 1: Core encoding and decoding microbenchmarks

Our `coding_benchmark_cases`.sh script measures pure encoding and decoding throughput in memory, excluding disk and network interference. Each test uses chunk=2048KB, or 2MB per data chunk. Encoding runs for 200 iterations; decoding throughput is averaged across all possible single-fragment-loss cases. We evaluate k=6,m=2, k=6,m=3, and k=8,m=3.

We also use Ceph's built-in `ceph_erasure_code_benchmark` tool to compare performance with AVX2 enabled and disabled and quantify the benefit of SIMD optimization.

Category 2: System-level cluster experiments

We use rados put/get to verify correctness and rados bench to measure end-to-end write and read bandwidth. Our `ec_recovery_compare`.sh monitoring script triggers OSD failures in a real cluster and records recovery time, peak bandwidth, and average bandwidth. Recovery experiments use development clusters with 10, 10, and 12 OSDs for k=6,m=2, k=6,m=3, and k=8,m=3, respectively.

The main metrics are core encoding and decoding throughput (GB/s), rados bench write and read bandwidth (MB/s), average and peak failure-recovery bandwidth (MB/s), and total recovery time (s).

4.4 Integration Validation

We validate correctness at three levels.

Level 1: Unit tests. Parameterized tests cover configurations including k=2,m=2, k=3,m=2, k=4,m=2, and k=6,m=3. For each configuration, the tests generate random input, encode it, reconstruct missing fragments through decoding, and compare the result with the original data.

Level 2: Toolchain validation. We inspect fragment contents with ceph-erasure-code-tool encode and decode. We write objects with rados put, read them with rados get, and compare their hashes with sha256sum. We use hexdump to verify that on-disk parity-fragment lengths match the values returned by `get_parity_chunk_size()`.

Level 3: Cluster robustness tests. On a multi-OSD Ceph development cluster, we run rados put/get and rados bench writes and reads with 4KB, 64KB, 1MB, and 4MB objects under different k, m configurations. We then kill an OSD to trigger recovery and check that PGs return to active+clean and data integrity is preserved. Table 4-1 reports the results.

Table 4-1 two-tone cluster robustness results

Configuration	Object sizes for writes and reads	rados bench write (MB/s)	rados bench read (MB/s)	Recovery successful
k=6, m=2	4KB/64KB/1MB/4MB	861.835	3471.62	Yes
k=6, m=3	4KB/64KB/1MB/4MB	786.389	3378.83	Yes
k=8, m=3	4KB/64KB/1MB/4MB	741.987	3472.7	Yes

All configurations pass end-to-end correctness checks. After failure recovery, the data's SHA-256 hashes exactly match those recorded at write time.

4.5 Core Encoding and Decoding Performance

4.5.1 Encoding Throughput for Large Chunks

Table 4-2 reports encoding throughput measured with `coding_benchmark_cases`.sh and chunk=2048KB.

Table 4-2 Core encoding throughput (chunk=2048KB; GB/s)

Configuration	two-tone	Jerasure	ISA-L	two-tone vs Jerasure
k=6, m=2	11.49	1.26	8.36	+812%
k=6, m=3	7.75	0.74	5.79	+945%
k=8, m=3	7.60	0.74	5.80	+927%

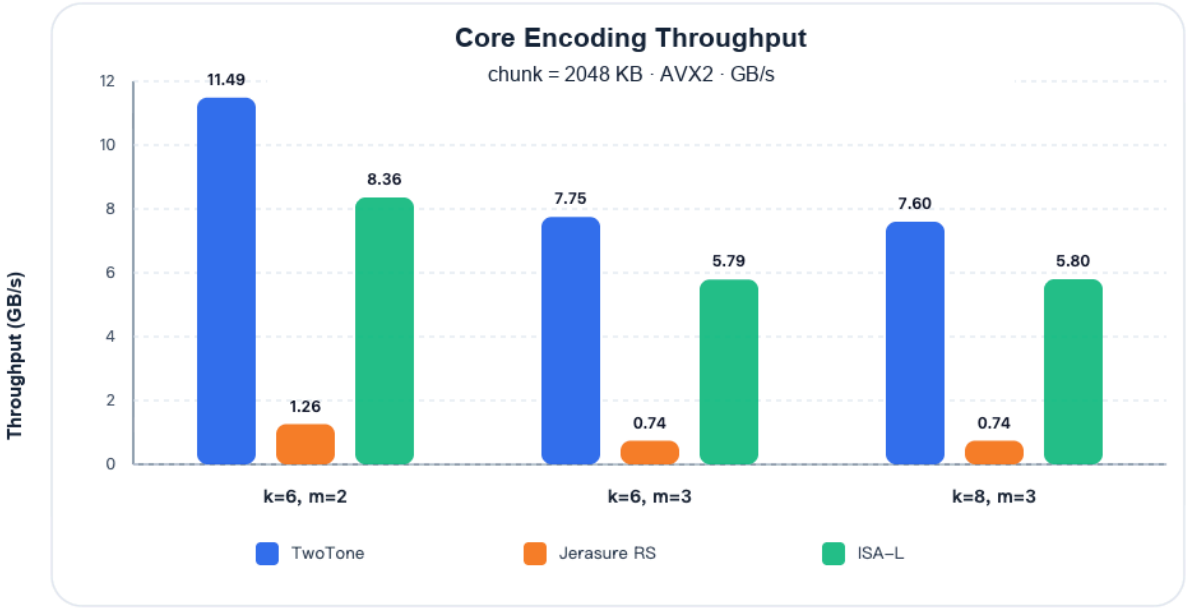


Figure 4-2 Core encoding throughput by EC plugin (chunk=2048KB)

Table 4-2 and Figure 4-2 show the following:

1. two-tone maintains a substantial encoding advantage over Jerasure RS. Across the three configurations, its throughput is approximately 9.1×, 10.5×, and 10.3× that of Jerasure RS, respectively. The Shift-XOR encoding path thus retains high throughput across the evaluated parameter combinations.

2. two-tone also outperforms ISA-L in encoding, by approximately 37.4%, 33.9%, and 31.0% across the three configurations. With AVX2 optimization, the implementation competes effectively with an industrial high-performance library as well as outperforming conventional GF-matrix-based schemes.

3. As the configuration changes from k=6,m=2 to k=8,m=3 with greater redundancy, encoding throughput declines for all plugins. Nevertheless, two-tone sustains 7.60 GB/s in the largest evaluated configuration, indicating that the precomputed layout and SIMD XOR kernel scale well to larger k,m combinations.

4.5.2 Decoding Throughput for Large Chunks

Table 4-3 reports decoding throughput averaged across all possible single-fragment-loss cases.

Table 4-3 Core decoding throughput (chunk=2048KB; GB/s)

Configuration	two-tone	Jerasure RS	ISA-L	two-tone vs Jerasure
k=6, m=2	17.89	2.30	15.94	+678%
k=6, m=3	13.67	1.83	12.06	+647%
k=8, m=3	13.23	1.73	12.15	+664%



Figure 4-3 Core decoding throughput by EC plugin (chunk=2048KB)

The decoding results show the following:

1. two-tone achieves decoding throughputs of 17.89, 13.67, and 13.23 GB/s across the three configurations, approximately 7.8×, 7.5×, and 7.6× the throughput of Jerasure RS. Its advantage is even more pronounced than in encoding.

2. two-tone also leads ISA-L by 8.9%-13.3% across the three configurations. The fast recovery path, tile-based cache optimization, and AVX2 XOR operations effectively offset the memory-access complexity introduced by variable-length shifted layouts.

3. two-tone performs best at k=6,m=2, reaching 17.89 GB/s, and consistently exceeds 13 GB/s at k=6,m=3 and k=8,m=3. These results indicate that the implementation adapts well to different redundancy configurations.

4.6 System-Level Storage and Recovery Performance

4.6.1 rados bench Write and Read Performance

In the cluster experiments, two-tone completes rados bench runs lasting 60 seconds with 4MB objects under all three configurations. Table 4-1 summarizes the results. Write throughput is 861.835, 786.389, and 741.987 MB/s; read throughput is 3471.62, 3378.83, and 3472.7 MB/s, respectively. For k=6,m=2, read bandwidth reaches 3471.62 MB/s, showing that support for variable-length fragments preserves Ceph's end-to-end read and write paths.

rados bench measures end-to-end performance, including EC computation, the network stack, coordination between primary and secondary OSDs, and disk I/O. Its values are therefore higher than those of the algorithm-only microbenchmarks. The two tests serve complementary purposes: microbenchmarks identify algorithmic bottlenecks, while system benchmarks establish usability under application-level operations.

4.6.2 Failure Recovery Performance

Recovery performance is a central measure of the practical value of an erasure code. Recovery combines reads from surviving fragments, reconstruction, and writeback, exposing the code's overall behavior in a real cluster.

Table 4-4 reports the complete recovery results for the three configurations. Figure 4-4 compares average recovery bandwidth.

Table 4-4 Failure recovery performance by plugin (4MB objects; real cluster)

Configuration	Plugin	Recovery time (s)	Peak bandwidth (MB/s)	Average bandwidth (MB/s)
k=6, m=2	two-tone	98	581	236
k=6, m=2	Jerasure RS	124	578	162
k=6, m=2	ISA-L	72	572	187
k=6, m=3	two-tone	82	274	123
k=6, m=3	Jerasure RS	121	276	80
k=6, m=3	ISA-L	75	259	119
k=8, m=3	two-tone	82	218	124
k=8, m=3	Jerasure RS	114	197	82
k=8, m=3	ISA-L	72	188	119

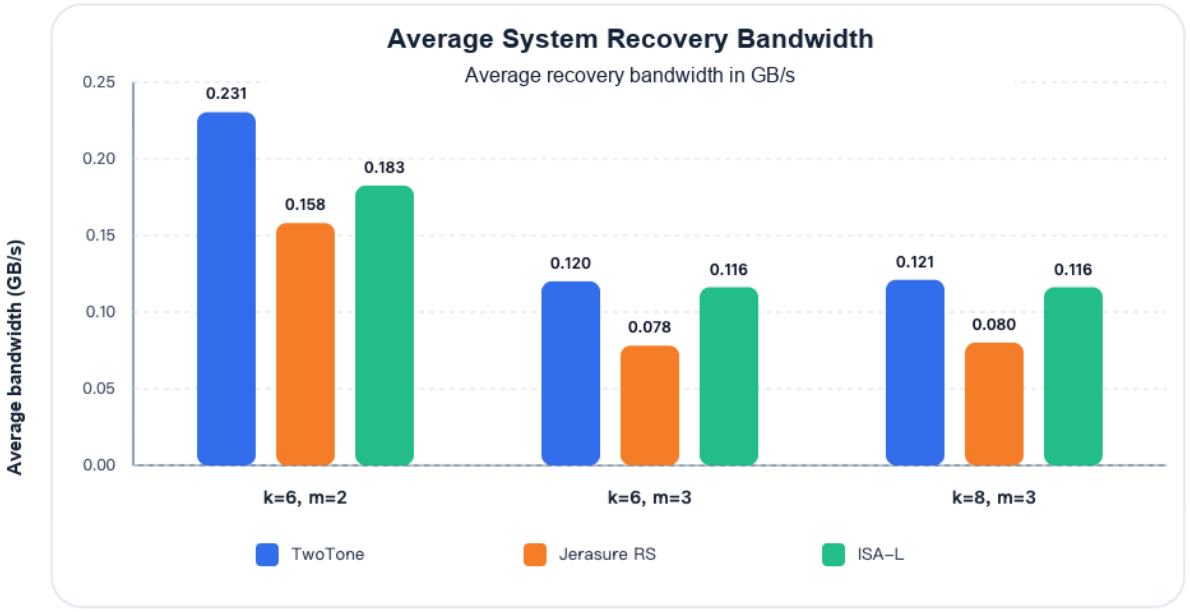


Figure 4-4 Average system recovery bandwidth by EC plugin

Table 4-4 and Figure 4-4 support the following conclusions:

two-tone substantially outperforms Jerasure RS under all three configurations:

k=6,m=2: average bandwidth is 236 versus 162 MB/s, a 45.7% increase, and recovery completes 26 seconds sooner.

k=6,m=3: average bandwidth is 123 versus 80 MB/s, a 53.8% increase, and recovery completes 39 seconds sooner.

k=8,m=3: average bandwidth is 124 versus 82 MB/s, a 51.2% increase, and recovery completes 32 seconds sooner.

two-tone's average recovery bandwidths of 236, 123, and 124 MB/s also exceed ISA-L's 187, 119, and 119 MB/s, respectively.

two-tone sustains average recovery bandwidth near 120 MB/s for the higher-redundancy k=6,m=3 and k=8,m=3 configurations. This suggests that the variable-length fragment compatibility mechanism continues to support stable system-level recovery as k and m increase.

Conclusion

This thesis implements an efficient distributed storage system based on two-tone codes. It covers algorithm implementation, Ceph plugin development, support for variable-length parity chunks, system-level optimization, and experimental evaluation.

For system integration, we address the main obstacles to using two-tone in a real distributed storage system: the equal-sized fragment assumption, alignment constraints for persistence, incorrect recovery-length inference, and less visible conflicts along the read and write paths. We add `get_parity_chunk_size()` to the plugin interface, correct five instances of the equal-sized fragment assumption in the OSD layer, and combine block-aligned padding with logical trimming. These changes allow two-tone to support the complete data lifecycle of writes, reads, benchmarking, and failure recovery.

For performance, layout precomputation, a fast recovery path, tile-based cache optimization, temporary buffer reuse, and AVX2 XOR operations increase core decoding throughput from approximately 0.1 GB/s to 13.23-17.89 GB/s. Across k=6,m=2, k=6,m=3, and k=8,m=3, two-tone substantially outperforms Jerasure RS in core encoding and decoding and exceeds ISA-L in the core microbenchmarks. In system-level recovery tests, its average bandwidths of 236, 123, and 124 MB/s improve on Jerasure RS by 45.7%-53.8% and exceed ISA-L on this metric.

The work contributes in two ways. First, it provides a Ceph adaptation framework for fragments of heterogeneous lengths, bridging the engineering gap between the theoretical two-tone code and deployment in a real system. The approach offers a reusable path for integrating other new codes, particularly Shift-XOR codes and schemes that generate variable-length fragments, into Ceph and similar open-source storage systems. Second, microbenchmarks, system benchmarks, and recovery experiments together demonstrate two-tone's competitiveness in coding efficiency, recovery performance, and practical usability.

Future work can reduce the effect of physical padding on network traffic and extend SIMD optimization to AVX-512 or ARM NEON. These improvements may make two-tone useful across larger and more diverse distributed storage deployments.

References

[1] Weil S A, Brandt S A, Miller E L, et al. Ceph: A Scalable, High-Performance Distributed File System[C]//Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2006: 307-320.

[2] Plank J S. Erasure Codes for Storage Applications[C]//Proceedings of the 4th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2005.

[3] Fu X, Wu C, Guo Y, et al. two-tone: A Shift-XOR Storage Code[C]//Proceedings of the IEEE International Symposium on Information Theory (ISIT). IEEE, 2021: 1234-1239.

[4] Reed I S, Solomon G. Polynomial Codes Over Certain Finite Fields[J]. Journal of the Society for Industrial and Applied Mathematics, 1960, 8(2): 300-304.

[5] Wicker S B, Bhargava V K. Reed-Solomon Codes and Their Applications[M]. New York: IEEE Press, 1994.

[6] Dimakis A G, Godfrey P B, Wu Y, et al. Network Coding for Distributed Storage Systems[J]. IEEE Transactions on Information Theory, 2010, 56(9): 4539-4551.

[7] Papailiopoulos D S, Dimakis A G. Locally Repairable Codes[J]. IEEE Transactions on Information Theory, 2014, 60(10): 5843-5855.

[8] Bloemer J, Kalfane M, Karp R, et al. An XOR-Based Erasure-Resilient Coding Scheme[R]. Berkeley: ICSI Technical Report TR-95-048, 1995.

[9] Fu X, Yang S, Xiao Z. Decoding and Repair Schemes for Shift-XOR Regenerating Codes[J]. IEEE Transactions on Information Theory, 2020, 66(8): 4862-4877.

[10] Fu X, Xiao Z, Yang S. Overhead-Free In-Place Recovery and Repair Schemes of XOR-Based Regenerating Codes[C]//Proceedings of the IEEE International Symposium on Information Theory (ISIT). IEEE, 2015: 1439-1443.

[11] Ghemawat S, Gobioff H, Leung S T. The Google File System[C]//Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2003: 29-43.

[12] Li B, et al. Erasure Coding Techniques for Faster Distributed Computing and Storage[J]. IEEE Network, 2018, 32(2): 36-43.

[13] Intel Corporation. Intel Storage Acceleration Library (ISA-L)[EB/OL]. [2026-04-21].

[14] Blaum M, Brady J, Bruck J, et al. EVENODD: An Efficient Scheme for Tolerating Double Disk Failures in RAID Architectures[J]. IEEE Transactions on Computers, 1995, 44(2): 192-202.

[15] Corbett P, English B, Goel A, et al. Row-Diagonal Parity for Double Disk Failure Correction[C]//Proceedings of the 3rd USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2004: 1-14.

[16] Huang C, Simitci H, Xu Y, et al. Erasure Coding in Windows Azure Storage[C]//Proceedings of the USENIX Annual Technical Conference (ATC). USENIX Association, 2012: 15-26.

[17] Plank J S, Luo J, Schuman C D, et al. A Performance Evaluation and Examination of Open-Source Erasure Coding Libraries with HDFS[C]//Proceedings of the 7th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2009: 253-265.

[18] Weil S A, Brandt S A, Miller E L, et al. CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data[C]//Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC). IEEE, 2006: 122.

Harbin Institute of Technology Undergraduate Thesis Declaration of Originality and Terms of Use

Declaration of Originality

I solemnly declare that the undergraduate thesis, Implementation of an Efficient Distributed Storage System Based on two-tone Codes, presents research that I conducted independently under my supervisor's guidance while pursuing a bachelor's degree at Harbin Institute of Technology. Except where references are explicitly cited, this thesis contains no research completed or published by others. Individuals and groups that made significant contributions to this work are explicitly acknowledged. The use of AI tools is explicitly identified in this thesis.



Author's signature: Date: May 19, 2026

Terms of Use

This undergraduate thesis was completed while the author pursued a bachelor's degree at Harbin Institute of Technology. Its intellectual property belongs to Harbin Institute of Technology. The following terms govern its use:

(1) The University may retain submitted undergraduate theses through photocopying, microfilming, or other reproduction methods and submit them to relevant authorities. (2) As needed, the University may include part or all of a thesis in databases for retrieval and provide corresponding reading services. (3) After graduation, the author must obtain the supervisor's consent before publishing academic papers or other outputs related to this thesis, and Harbin Institute of Technology must be listed as the first affiliation.

Confidential theses remain subject to the applicable confidentiality rules during the restriction period. These terms of use apply after declassification.

I understand the terms of use for this undergraduate thesis and will comply with the relevant provisions.



Author's signature: Date: May 19, 2026

Supervisor's signature: Date: May 19, 2026



Acknowledgments

This project took almost a year, from the initial proposal and midterm review to the final thesis. The work involved algorithm implementation, system debugging, performance optimization, and verification of the experimental results.

I sincerely thank my supervisor, Ximing Fu, for his continued guidance on the research direction, technical approach, and writing. His advice helped this work develop from a functional prototype into an implementation with practical systems value. Integrating two-tone with Ceph proved far more complex than I had anticipated. His encouragement and guidance helped me persist in identifying the root cause of each crash.

I thank the Ceph open-source community for its codebase, documentation, and testing tools, which greatly reduced the effort required to explore the implementation. two-tone also builds on sustained research into XOR-based erasure coding, and I thank Fu and his collaborators for their pioneering work.

I thank my roommates and classmates for their discussions and assistance throughout this project. My learning and growth over four undergraduate years depended on the support and companionship of those around me.